

Rapport de stage

Méthodes hybrides pour l'ordonnancement sous contraintes de ressources complexes

August 12, 2026

Stagiaire :

Alex TSAN alex.tsan@utoulouse.fr

Tuteurs de stage :

Christian ARTIGUES

Carla JUVIN

Pierre LOPEZ

Mot clés : Recherche opérationnelle; Programmation par contraintes; Décomposition de Benders; Programmation linéaire; Ordonnancement multi-compétences

Abstract :

L'optimisation de l'ordonnancement de projets avec ressources multi-compétences et flexibilité d'affectation (MS-RCPSP-AF) pose d'importants défis combinatoires. Ce stage vise à comparer différents paradigmes de résolution exacte pour ce problème. Nous analysons et évaluons les performances de modèles monolithiques en Programmation Linéaire en Nombres Entiers (MILP) et en Programmation par Contraintes (CP) face à une méthode de Décomposition de Benders basée sur la Logique (LBBD). Nous présentons d'abord une formulation en MILP indexée par le temps et un modèle en CP discret découpant les tâches en sous-intervalles unitaires pour modéliser le

remplacement dynamique des opérateurs. Pour le modèle en CP, nous étudions également une formulation alternative basée sur la contrainte de cardinalité globale (GCC) et l'ajout de contraintes de bris de symétrie. Face aux limites physiques et combinatoires de ces modèles monolithiques, nous implémentons un algorithme de LBBD. Cette méthode découple l'ordonnancement global, traité par un problème maître en CP, de l'affectation des travailleurs à chaque période, résolue par des algorithmes de flot maximum. Les résultats expérimentaux obtenus sur deux benchmarks de la littérature montrent que la décomposition LBBD résout entre 94 % et 97 % des instances en quelques secondes, surpassant nettement les approches monolithiques.

Contents

Introduction	1
Contexte et présentation du LAAS-CNRS	1
Énoncé du problème et objectifs du stage	2
Annonce du plan	4
1 Programmation linéaire en nombres entiers	6
1.1 Structure d'un modèle MILP	6
1.2 Mécanismes de résolution	6
1.3 Application au MS-RCPSP-AF	7
1.3.1 Ensembles	7
1.3.2 Paramètres	7
1.3.3 Variables de décision	8
1.3.4 Fonction objectif	8
1.3.5 Contraintes	8
1.4 Limites de la formulation MILP	9
1.4.1 Explosion du nombre de variables avec l'horizon temporel	9
1.4.2 Faiblesse de la relaxation continue	9
1.4.3 Linéarisation des relations logiques	9
2 Programmation par Contraintes	11
2.1 Formalisme des Problèmes de Satisfaction de Contraintes (CSP)	11
2.2 Mécanismes de résolution : Propagation et Recherche	12
2.3 Primitives d'ordonnancement de CP Optimizer	12
2.3.1 Variables d'intervalles (Interval Variables)	12
2.3.2 Contraintes temporelles	12
2.3.3 Ressources disjonctives et cumulatives	13
2.4 Modélisation CP monolithique pour le MS-RCPSP-AF	13
2.4.1 Ensembles et paramètres	13
2.4.2 Variables de décision	13
2.4.3 Formulation mathématique	14
2.4.4 Explication des contraintes	14

2.4.5	Contraintes redondantes de capacité globale	15
2.5	Variante : Modélisation par Contrainte de Cardinalité Globale (GCC)	15
2.5.1	Variables de décision du modèle GCC	15
2.5.2	Formulation du modèle GCC et contraintes d'affectation	16
2.6	Contraintes de bris de symétrie	16
2.7	Limites des modèles CP monolithiques	17
2.7.1	Explosion dimensionnelle du nombre de variables	17
2.7.2	Perte de la structure globale d'ordonnancement et faiblesse de la propagation	18
2.7.3	Symétries et inefficacité des contraintes de bris de symétrie	18
3	Décomposition de Benders basée sur la logique	19
3.1	Principes de la Décomposition de Benders basée sur la Logique appliquée au MS- RCPSP-AF	19
3.2	Problème maître (Ordonnancement)	20
3.2.1	Formulation mathématique	20
3.3	Sous-problème d'affectation	21
3.3.1	Formulation sous forme de flot maximum (Max-Flow)	21
3.4	Génération des coupes de Benders	22
3.4.1	Recherche de coupe minimale sur le graphe biparti	23
3.4.2	Formulation de la coupe injectée	24
4	Évaluation Expérimentale	25
4.1	Protocole expérimental et instances de test	25
4.1.1	Environnement de test	25
4.1.2	Premier jeu de données : Set 1a de la bibliothèque MSPSP-InstLib	25
4.1.3	Autre jeu de données : Subset 1 de MSLIB	26
4.2	Comparaison des approches sur le Set 1a de MSPSP-InstLib	26
4.2.1	Analyse des taux de résolution et des temps de calcul	27
4.2.2	Comparaison directe sur les instances faciles (communes)	27
4.3	Visualisations graphiques pour le Set 1a	28
4.3.1	Impact du nombre de travailleurs sur les performances de résolution	28
4.4	Résultats et comparaison sur le Subset 1 de MSLIB1	29
4.4.1	Analyse des taux de résolution et des temps de calcul	30

4.4.2	Comparaison directe sur les instances faciles (communes)	30
4.4.3	Comparaison du comportement MSLIB vs Set 1a	30
4.5	Visualisations graphiques pour le Subset 1	31
Conclusion et Perspectives		32

Introduction

Contexte et présentation du LAAS-CNRS

Ce document présente les travaux de stage de fin d'année de Master 1 Informatique, parcours Intelligence Artificielle : Fondements et Applications (IAFA) à l'Université de Toulouse. Ce stage s'est déroulé au LAAS-CNRS (Laboratoire d'Analyse et d'Architecture des Systèmes) à Toulouse. Fondé en 1968, le LAAS est une Unité Propre de Recherche (UPR 8001) du CNRS, rattachée aux Instituts Sciences Informatiques et Ingénierie. Le laboratoire est conventionné avec les principaux établissements d'enseignement supérieur de la région : l'Université de Toulouse, l'INSA Toulouse, Toulouse INP et l'ISAE-SUPAERO. Il regroupe environ 650 personnes, comprenant des chercheurs, enseignants-chercheurs, ingénieurs, techniciens, doctorants et post-doctorants.

Les activités du LAAS-CNRS s'organisent autour de quatre thèmes principaux : l'Automatique, l'Informatique, les Micro et Nano Systèmes, et la Robotique. Ce spectre permet de lier le développement matériel aux algorithmes.

La recherche au LAAS-CNRS s'articule autour de six départements : DO, GE, HOPES, MNBT, RISC et ROB.

- **Décision et Optimisation** : Le département DO développe des théories mathématiques et des outils algorithmiques pour la prise de décision et la commande de systèmes complexes. Ses recherches s'articulent autour de deux approches complémentaires : la synthèse de lois de décision via des techniques d'optimisation et la résolution de problèmes d'optimisation combinatoire discrets. Ses travaux trouvent des applications dans les réseaux électriques, les transports et l'industrie du futur. C'est dans ce cadre, axé sur l'optimisation combinatoire sous contraintes, que s'intègrent les recherches de l'équipe ROC en ordonnancement et planification.
- **Gestion de l'Energie** : Le département GE se consacre à l'optimisation de l'efficacité énergétique, à la robustesse et à la fiabilité des architectures de conversion de puissance. Pour répondre aux défis de la transition écologique, de la décentralisation des énergies renouvelables et de l'électrification des transports, il conçoit des composants électroniques miniatures et des systèmes de gestion de l'énergie. Le département valide ses prototypes au sein du bâtiment expérimental connecté ADREAM.
- **Systèmes Hyperfréquences et Photoniques** : Le département HOPES explore la convergence entre l'électronique de pointe et la photonique pour concevoir les briques de base de l'Internet des objets (IoT) et des futurs systèmes cyber-physiques. Ses recherches visent à créer des composants miniatures et à faible consommation d'énergie, capables de communiquer avec leur environnement.
- **Micro Nano Bio Technologies** : Le département MNBT combine des approches fondamentales et appliquées pour concevoir des nanomatériaux et des nano-objets, puis les intégrer dans des dispositifs complexes. Ses activités ciblent trois défis : les technologies pour le numérique, les technologies pour la santé et la surveillance environnementale.

- **Réseaux, Informatique, Systèmes de Confiance** : Le département RISC cible la maîtrise des architectures informatiques et des réseaux de communication distribués. Ses recherches se focalisent sur la cybersécurité, l'orchestration dynamique des ressources cloud et le développement d'une intelligence artificielle digne de confiance et frugale en énergie.
- **Robotique** : Le département ROB déploie une recherche dédiée à l'autonomie motrice et cognitive des systèmes physiques opérant dans des environnements réels et non maîtrisés. Ses domaines d'expertise couvrent la planification de mouvements pour les structures anthropomorphes, le couplage entre la perception et l'action, ainsi que la gestion des interactions entre le robot, l'Homme et son milieu.

Ce stage s'est déroulé au sein du département Décision et Optimisation (DO), dans l'équipe ROC (Recherche Opérationnelle, Optimisation Combinatoire et Contraintes) dirigée par Christian Artigues. L'équipe ROC se consacre à la résolution de problèmes d'optimisation combinatoire discrets complexes. Son objectif est de concevoir des modèles mathématiques et des algorithmes pour trouver les meilleures décisions possibles face à un grand nombre de choix. Ses recherches s'organisent autour de trois piliers méthodologiques :

- **La programmation linéaire** : Formulations mathématiques et approches polyédrales.
- **La programmation par contraintes** : Exploitation de structures logiques et de relations temporelles.
- **Les méthodes hybrides et décompositions** : Couplage de plusieurs paradigmes pour traiter les problèmes de grande taille.

Énoncé du problème et objectifs du stage

Le problème abordé s'inscrit dans le domaine de l'optimisation combinatoire et de l'ordonnancement sous contraintes de ressources. Plus précisément, il s'agit du problème d'ordonnancement de projet sous contraintes de ressources multi-compétences avec flexibilité d'affectation (Multi-Skill Resource-Constrained Project Scheduling Problem with Allocation Flexibility, ou MS-RCPSP-AF).

Dans sa formulation classique (RCPSP), l'ordonnancement de projet sous contraintes de ressources consiste à planifier des tâches liées par des contraintes de précédences tout en respectant la capacité limitée de ressources cumulatives (ressources disponibles en quantité globale constante à chaque instant, comme un parc de machines), avec pour objectif de minimiser la durée totale du projet (makespan). La variante MS-RCPSP (MS pour Multi-Skill) gère des ressources humaines avec des profils hétérogènes : les opérateurs maîtrisent une ou plusieurs compétences spécifiques. Les besoins d'une tâche sont définis par des exigences en compétences. Dans ce cadre traditionnel, l'affectation est rigide, ce qui implique que les opérateurs sélectionnés restent sur une tâche du début à la fin.

L'introduction de la flexibilité d'affectation (MS-RCPSP-AF), concept introduit par Juvin et al. [1], lève cette hypothèse en autorisant la rotation ou le remplacement des opérateurs au cours de

l'exécution d'une tâche. La seule condition requise est que les exigences en compétences de la tâche soient couvertes à chaque période.

Pour illustrer l'intérêt de cette flexibilité d'allocation (AF), la Figure 1 compare deux scénarios d'exécution d'un projet composé de trois tâches (Tâche A de durée 10 requérant s_1 , Tâche B de durée 5 requérant s_2 , et Tâche C de durée 5 requérant s_3) à réaliser par deux travailleurs (W_1 maîtrisant $\{s_1, s_2\}$ et W_2 maîtrisant $\{s_1, s_3\}$) :

- **Scénario 1 (Sans Flexibilité) :** L'affectation est rigide. Le planning présenté est un ordonnancement optimal conduisant à un Makespan de 15 heures. La Tâche B est exécutée de 0 à 5 par W_1 , et la Tâche C de 5 à 10 par W_2 . La Tâche A commence à l'instant 5 et est effectuée par W_1 pour se terminer à l'instant 15. (*Remarque : un autre ordonnancement optimal consisterait à planifier la Tâche A de 0 à 10 par W_2 , la Tâche B de 0 à 5 par W_1 et la Tâche C de 10 à 15 par W_2*).
- **Scénario 2 (Avec Flexibilité) :** Un remplacement est possible. La Tâche A débute dès l'instant 0, prise en charge par W_2 pour sa première moitié. À l'instant 5, la Tâche C doit démarrer et requiert W_2 pour sa compétence s_3 . Grâce à la flexibilité, W_2 cède sa place sur la Tâche A à W_1 (qui vient de terminer la Tâche B). La Tâche A se poursuit sans interruption et se termine à l'instant 10, tout comme la Tâche C. Le Makespan est ainsi réduit de 15h à 10h.

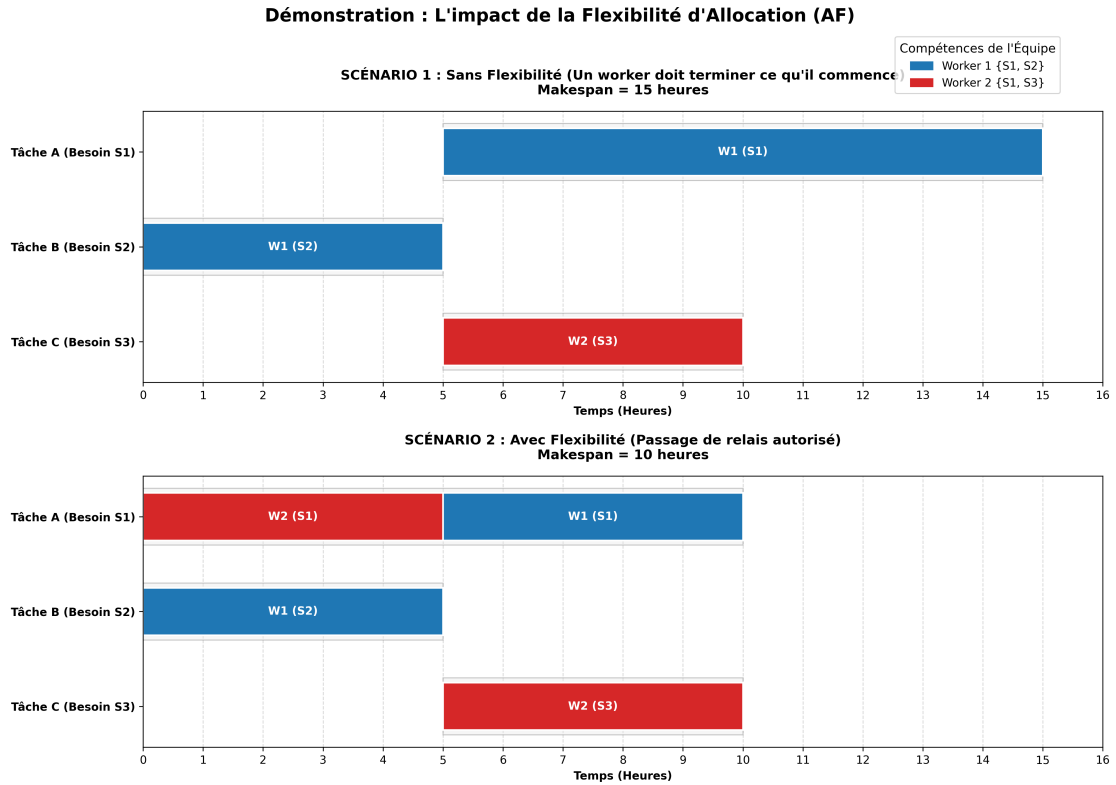


Figure 1: Illustration de l'impact de la flexibilité d'affectation (AF) sur le Makespan d'un projet

Si cette flexibilité permet d’optimiser l’usage des ressources et de réduire le makespan, elle augmente la complexité du problème. Contrairement aux approches classiques où l’affectation est fixe pour toute la durée d’une tâche, la flexibilité impose de valider la faisabilité de l’affectation à chaque unité de temps. Cela génère une combinatoire importante, renforcée par des symétries d’affectation lorsque les opérateurs partagent des compétences identiques. Les solveurs peinent alors à élaguer l’arbre de recherche.

La littérature de la recherche opérationnelle s’est largement intéressée au problème classique d’ordonnancement de projet sous contraintes de ressources (RCPSP). L’intégration de ressources multi-compétences, introduite par Bellenguez-Morineau et Néron [2, 3], a suscité un intérêt croissant, comme le montre la revue de littérature de Snauwaert et Vincke [4]. Les approches exactes traditionnelles reposent souvent sur la programmation linéaire en nombres entiers (MILP), étudiée par Almeida et al. [5], mais ces formulations présentent des limites de passage à l’échelle lorsque l’horizon s’allonge. Pour y remédier, des approches basées sur la génération de colonnes (Branch-and-Price) ont été explorées [6, 7]. Parallèlement, la Programmation par Contraintes (CP) s’est imposée comme une alternative pour modéliser des contraintes complexes [8, 9]. Néanmoins, ces modèles monolithiques atteignent leurs limites sur des instances de grande taille en raison de la combinatoire induite par la flexibilité. C’est pourquoi la recherche s’oriente vers des méthodes de décomposition hybrides. La Décomposition de Benders basée sur la Logique (LBBD), introduite par Hooker [10, 11] et analysée par Cire et al. [12], apparaît comme l’une des méthodes les plus adaptées. Contrairement aux schémas de Benders classiques qui délèguent l’ordonnancement au sous-problème, certaines approches d’ordonnancement multi-compétences inversent ce schéma en résolvant l’ordonnancement global dans le problème maître en CP et l’affectation dans le sous-problème. Cette inversion, introduite par Yi-fan et al. [13], a été étendue par Juvin et al. [1] pour modéliser la flexibilité d’affectation (AF) dans le cadre du MS-RCPSP-AF, permettant le remplacement des travailleurs au cours de l’exécution des tâches.

Annnonce du plan

Ce manuscrit est structuré en quatre chapitres principaux :

Le Chapitre 1 introduit la modélisation mathématique du problème sous forme de programme linéaire en nombres entiers (MILP). Cette approche permet de formaliser les contraintes d’affectation et de précédence, tout en mettant en évidence les limites de ce type de formulation.

Le Chapitre 2 explore la modélisation en Programmation par Contraintes (CP). Nous présentons la formulation classique par sous-intervalles unitaires, une variante reposant sur la contrainte de cardinalité globale (GCC) et des contraintes de bris de symétrie, en analysant leurs limites calculatoires respectives.

Le Chapitre 3 présente la Décomposition de Benders basée sur la Logique (LBBD). Ce chapitre détaille la génération de coupes logiques et le couplage entre un problème maître en CP pour l’ordonnancement global et un Sous-Problème d’affectation résolu par flot maximum.

Le Chapitre 4 présente l'évaluation expérimentale des approches sur les instances des bibliothèques MSPSP-InstLib et MSLIB, confirmant la supériorité de la décomposition face à l'explosion combinatoire subie par les modélisations monolithiques. Enfin, un aperçu sur les perspectives de recherche conclut ce manuscrit.

1 Programmation linéaire en nombres entiers

La Programmation Linéaire en Nombres Entiers (PLNE, ou MILP en anglais) est un outil d'optimisation classique en Recherche Opérationnelle pour résoudre les problèmes de décision complexes. La MILP permet de modéliser des problèmes NP-difficiles [14] en combinant une formulation mathématique avec la puissance des solveurs numériques [15].

Dans le cadre du RCPSP et du MS-RCPSP, la MILP a constitué l'une des premières approches de résolution exacte [16]. Les formulations linéaires décrivent les relations temporelles entre tâches et les contraintes d'affectation. En particulier, les formulations indexées par le temps (où l'horizon temporel est discrétisé en unités de temps) servent de référence dans la littérature pour évaluer d'autres paradigmes [5]. Ces modèles linéaires sont aussi utilisés dans des approches hybrides, que ce soit lors d'une décomposition de Benders [17, 18] ou pour la génération de colonnes [6, 7].

Ce chapitre présente les bases théoriques de la MILP, les mécanismes de résolution des solveurs, puis détaille la modélisation mathématique du problème du MS-RCPSP-AF inspirée de Polo-Mejía et al. [9, 19, 20].

1.1 Structure d'un modèle MILP

Un programme linéaire en nombres entiers comporte trois éléments principaux :

Les variables de décision : Ce sont les inconnues à déterminer pour résoudre le problème. En MILP, ces variables prennent des valeurs entières, et sont souvent binaires (0 ou 1) pour représenter des choix de type oui/non.

La fonction objectif : Une fonction linéaire à optimiser (généralement la minimisation de la date de fin de projet, le makespan). Elle s'exprime comme une somme de variables pondérées par des coefficients, mais dans notre modèle, elle se réduit à la minimisation d'une unique variable représentée par le makespan.

Les contraintes : Un ensemble d'équations et d'inégalités linéaires qui limitent l'espace des solutions réalisables (capacités, précédences ou limites de ressources). Les multiplications de variables ou les conditions logiques directes y sont interdites pour conserver la linéarité.

1.2 Mécanismes de résolution

Contrairement à la programmation linéaire continue, qui se résout en pratique en temps polynomial (bien que l'algorithme du Simplexe ait une complexité exponentielle dans le pire des cas), la contrainte d'intégralité des variables rend la MILP NP-difficile. L'espace des solutions réalisables se limite alors à un ensemble de points discrets.

Pour explorer cet espace, les solveurs modernes (comme Gurobi ou IBM ILOG CPLEX) combinent la programmation linéaire continue avec des algorithmes d'exploration arborescente, principalement la méthode de Séparation et Évaluation (Branch-and-Bound) et le Branch-and-Cut.

La relaxation continue : À chaque nœud de l'arbre de recherche, le solveur résout le problème en omettant la contrainte d'intégralité des variables, ce qui fournit une borne inférieure pour un problème de minimisation.

Branching : Si une variable devant être entière prend une valeur fractionnaire (par exemple 2,4) dans la relaxation continue, le solveur sépare l'espace de recherche en créant deux sous-nœuds : l'un imposant que la variable soit inférieure ou égale à 2, et l'autre supérieure ou égale à 3.

L'élagage (Bounding and Pruning) : L'arbre est exploré de manière itérative. Si la borne continue calculée à un nœud est moins bonne qu'une solution entière déjà connue (par exemple plus grand que la borne supérieure pour un problème de minimisation), le solveur coupe ce nœud et sa descendance pour éviter une recherche inutile.

1.3 Application au MS-RCPSP-AF

Nous formalisons ici le MS-RCPSP-AF sous forme de modèle MILP pour poser les bases mathématiques du problème avant de détailler la modélisation par programmation par contraintes.

1.3.1 Ensembles

- $\mathcal{A}$: Ensemble des activités à planifier.
- $\mathcal{L}$: Ensemble des différentes compétences.
- $\mathcal{W}$: Ensemble des ressources multi-compétences (travailleurs).
- $\mathcal{CR}$: Ensemble des ressources cumulatives.
- $\mathcal{T}$: Horizon temporel, $\mathcal{T} = \{1, \dots, T\}$, indexé par t .
- E : Ensemble des contraintes de précédence.

1.3.2 Paramètres

- p_i : Temps de traitement de l'activité i .
- B_k : Capacité de la ressource k .
- $b_{i,k}$: Quantité de ressource k requise pour l'activité i .
- $a_{i,l}$: Nombre de travailleurs maîtrisant la compétence l requis pour l'activité i .
- $m_{w,l}$: Égal à 1 si le travailleur w maîtrise la compétence l , 0 sinon.
- q_i : Nombre minimum de travailleurs requis pour l'activité i .

- r_i, d_i : Date de disponibilité et date d'échéance autorisées pour l'activité i .

1.3.3 Variables de décision

- $x_{i,t} \in \{0, 1\}$: Égal à 1 si l'activité i est exécutée à l'instant t , 0 sinon.
- $z_{w,i,l,t} \in \{0, 1\}$: Égal à 1 si le travailleur w est affecté à l'activité i pour exercer la compétence l à l'instant t , 0 sinon.

1.3.4 Fonction objectif

L'objectif est de minimiser la durée totale d'exécution du projet (*Makespan*) :

$$\min \quad C_{\max} \quad (1)$$

1.3.5 Contraintes

Unicité d'affectation : Un travailleur peut être affecté au maximum à une seule activité et une seule compétence peut être utilisée à un instant t .

$$\sum_{i \in \mathcal{A}} \sum_{l \in \mathcal{L}} z_{w,i,l,t} \leq 1 \quad \forall w \in \mathcal{W}, \forall t \in \mathcal{T} \quad (2)$$

Validation de la maîtrise des compétences : Un travailleur ne peut utiliser une compétence que s'il la maîtrise.

$$z_{w,i,l,t} \leq m_{w,l} \quad \forall w \in \mathcal{W}, \forall i \in \mathcal{A}, \forall l \in \mathcal{L}, \forall t \in \mathcal{T} \quad (3)$$

Couverture des exigences en compétences : Si une activité est active ($x_{i,t} = 1$), le nombre d'opérateurs affectés à la compétence l doit couvrir le besoin requis $a_{i,l}$.

$$\sum_{w \in \mathcal{W}} z_{w,i,l,t} \geq a_{i,l} \cdot x_{i,t} \quad \forall l \in \mathcal{L}, \forall i \in \mathcal{A}, \forall t \in \mathcal{T} \quad (4)$$

Respect de l'effectif minimum requis pour une activité : Le nombre total d'affectations pour l'activité i à l'instant t doit être supérieur ou égal à l'exigence q_i .

$$\sum_{w \in \mathcal{W}} \sum_{l \in \mathcal{L}} z_{w,i,l,t} \geq q_i \cdot x_{i,t} \quad \forall i \in \mathcal{A}, \forall t \in \mathcal{T} \quad (5)$$

Respect des durées de traitement : Chaque activité doit s'exécuter pendant exactement p_i périodes sur son intervalle de temps admissible.

$$\sum_{t=r_i}^{d_i} x_{i,t} = p_i \quad \forall i \in \mathcal{A} \quad (6)$$

Contraintes de précédence : Une activité successeur j ne peut pas démarrer tant que son activité

prédécesseur i n'est pas achevée.

$$p_i \cdot (1 - x_{j,t}) \geq \sum_{t'=t}^T x_{i,t'} \quad \forall (i, j) \in E, \forall t \in \mathcal{T} \quad (7)$$

Capacité des ressources matérielles cumulatives : L'utilisation des ressources matérielles cumulatives à l'instant t est limitée à leur capacité globale B_k .

$$\sum_{i \in \mathcal{A}} b_{i,k} \cdot x_{i,t} \leq B_k \quad \forall k \in \mathcal{CR}, \forall t \in \mathcal{T} \quad (8)$$

Définition de la borne inférieure du Makespan

$$C_{\max} \geq t \cdot x_{i,t} \quad \forall i \in \mathcal{A}, \forall t \in \mathcal{T} \quad (9)$$

Cette contrainte lie la variable $C_{\max}$ au déroulement du projet. Le modèle étant indexé par le temps, si une activité i s'exécute à l'instant t ($x_{i,t} = 1$), l'inéquation force le $C_{\max}$ à être supérieur ou égal à cet instant t . Lors de la minimisation de la fonction objectif, la variable $C_{\max}$ prend la valeur du dernier instant t où une tâche est active, ce qui correspond à la date de fin globale.

1.4 Limites de la formulation MILP

Le modèle MILP fournit une formulation exacte du problème du MS-RCPSP-AF, mais présente des limites pratiques pour la résolution d'instances de taille moyenne ou grande.

1.4.1 Explosion du nombre de variables avec l'horizon temporel

L'utilisation de variables indexées par le temps ($x_{i,t}$ et $z_{w,i,l,t}$) rend la taille du modèle dépendante de l'horizon temporel T . Le nombre de variables d'affectation $z_{w,i,l,t}$ s'élève à $|\mathcal{W}| \times |\mathcal{A}| \times |\mathcal{L}| \times T$. Si T est grand, le solveur doit gérer un grand nombre de variables binaires, ce qui sature la mémoire et rend la phase de séparation (*Branching*) longue et difficile.

1.4.2 Faiblesse de la relaxation continue

Les formulations indexées par le temps pour l'ordonnancement présentent souvent une relaxation continue faible. Lorsque les variables binaires $x_{i,t}$ et $z_{w,i,l,t}$ sont relâchées dans l'intervalle $[0, 1]$, le solveur linéaire trouve des solutions fractionnaires où les tâches sont fragmentées dans le temps. La borne inférieure fournie par cette relaxation est alors éloignée de la valeur entière optimale, ce qui oblige le solveur à explorer un grand nombre de nœuds dans l'arbre de recherche.

1.4.3 Linéarisation des relations logiques

Conserver un formalisme linéaire impose de linéariser les contraintes logiques. Pour les contraintes de précédence (7) :

$$p_i \cdot (1 - x_{j,t}) \geq \sum_{t'=t}^T x_{i,t'} \quad \forall (i, j) \in E, \forall t \in \mathcal{T}$$

Si le successeur j s'exécute à l'instant t ($x_{j,t} = 1$), le membre de gauche s'annule, forçant le prédécesseur i à être inactif à partir de cet instant t . Si j n'est pas actif à l'instant t ($x_{j,t} = 0$), la contrainte devient triviale ($p_i \geq \sum x_{i,t'}$) car la somme des périodes d'activité de i ne dépasse pas sa durée p_i . Cette linéarisation doit être écrite pour chaque période t et chaque arc du graphe de précedence E , ce qui augmente le nombre de contraintes à gérer par le solveur.

2 Programmation par Contraintes

La Programmation par Contraintes (PPC, ou CP en anglais) est un paradigme d'optimisation exacte adapté à la résolution de problèmes combinatoires comme la planification et l'ordonnancement. Contrairement à la programmation linéaire, ce paradigme permet d'exprimer des contraintes non linéaires, logiques ou globales.

En CP, l'utilisateur formule le problème sous forme de contraintes, et le processus de recherche de solutions et de preuve d'optimalité est pris en charge par un solveur générique. Ce solveur repose sur le couplage entre la propagation de contraintes et l'exploration arborescente. Les contraintes globales encapsulent des structures combinatoires récurrentes (telles que l'ordonnancement de tâches ou l'affectation), et leur sont associés des algorithmes de filtrage (propagateurs). Pour l'ordonnancement sous contraintes cumulatives, ces propagateurs utilisent des algorithmes comme le *timetabling* [21], le *cumulative edge-finding* [22] ou le raisonnement énergétique [23].

L'application de la CP au problème de l'ordonnancement de projet multi-compétences (MSPSP) qui n'est autre que la version sans contraintes de ressources cumulatives du MS-RCPSP, a donné lieu à des travaux notables. Des approches basées sur CP/SAT, le *lazy clause generation* et l'apprentissage de no-goods ont été développées par Young et al. [8]. Pour le MS-RCPSP avec préemption et contraintes de ressources, Polo-Mejía et al. [9, 19] ont proposé des modèles en CP exploitant les variables d'intervalles de solveurs comme *IBM ILOG CP Optimizer*. Par ailleurs, des travaux comme ceux de Carlier et Néron [24] ou de Baptiste et Bonifas [25] ont montré l'intérêt d'ajouter des enveloppes cumulatives redondantes pour accélérer la convergence.

Ce chapitre présente le formalisme général des CSP, décrit les primitives d'ordonnancement de CP Optimizer, formule les modèles CP monolithiques pour le MS-RCPSP-AF, puis analyse ses limites combinatoires.

2.1 Formalisme des Problèmes de Satisfaction de Contraintes (CSP)

Un CSP est défini par un triplet (X, D, C) où :

- $X = \{x_1, x_2, \dots, x_n\}$ est l'ensemble des variables de décision.
- $D = \{D(x_1), D(x_2), \dots, D(x_n)\}$ représente l'ensemble des domaines associés (les valeurs admissibles pour chaque variable, généralement des entiers).
- $C = \{c_1, c_2, \dots, c_m\}$ est l'ensemble des contraintes qui limitent les combinaisons de valeurs possibles pour les variables.

L'objectif est d'affecter à chaque variable x_i une valeur de son domaine $D(x_i)$ en satisfaisant toutes les contraintes de C .

2.2 Mécanismes de résolution : Propagation et Recherche

Pour explorer l'espace des solutions, les solveurs de CP (comme *IBM ILOG CP Optimizer*) couplent deux mécanismes :

1. **La propagation de contraintes (filtrage)** : À chaque contrainte $c_j \in C$ est associé un propagateur. Lorsqu'une décision restreint le domaine d'une variable, le propagateur évalue la cohérence locale (comme l'arc-cohérence) et élimine des domaines des autres variables connectées les valeurs devenues incompatibles.
2. **La recherche arborescente (exploration)** : Si la propagation ne suffit pas à obtenir une solution unique, le solveur effectue un branchement (*branching*) : il choisit une variable à domaine multiple, la fixe à une valeur, puis relance la propagation. Si une contrainte est violée (un domaine devient vide), le solveur effectue un retour arrière (*backtracking*) pour essayer une autre branche.

2.3 Primitives d'ordonnancement de CP Optimizer

Pour modéliser l'ordonnancement, les solveurs comme *IBM ILOG CP Optimizer* disposent de primitives de haut niveau qui évitent de discrétiser le temps de façon externe.

2.3.1 Variables d'intervalles (Interval Variables)

Une variable d'intervalle, notée a , représente une tâche ou une partie de tâche définie par un début $a.start$, une fin $a.end$, et une durée $a.size$, liés par la relation $a.end = a.start + a.size$. De plus, une variable d'intervalle peut être **optionnelle** : le solveur décide alors de sa présence ou non dans la solution ($a.present \in \{0, 1\}$). Si l'intervalle est absent, ses attributs de temps sont ignorés par les contraintes associées.

2.3.2 Contraintes temporelles

Les relations de précédence et de synchronisation entre activités s'expriment avec des contraintes spécifiques :

- **endBeforeStart(a, b, z)** : Impose que le début de l'intervalle b soit postérieur ou égal à la fin de l'intervalle a , avec un délai de transition z :

$$b.start \geq a.end + z \quad (10)$$

Cette contrainte n'est active que si a et b sont présents.

- **startAtEnd(a, b)** : Impose que l'intervalle a commence exactement au moment où l'intervalle b se termine :

$$a.start = b.end \quad (11)$$

Cette contrainte sert à modéliser des jonctions temporelles sans interruption.

- **span($a, \{b_1, \dots, b_k\}$)** : Force l'intervalle a à couvrir l'ensemble des intervalles présents du groupe $\{b_1, \dots, b_k\}$. Ainsi, a débute au début du premier intervalle actif et se termine à la fin

du dernier intervalle actif du groupe :

$$a.start = \min_{1 \leq i \leq k} \{b_i.start \mid b_i.present = 1\} \quad (12)$$

$$a.end = \max_{1 \leq i \leq k} \{b_i.end \mid b_i.present = 1\} \quad (13)$$

- **alternative(a, {b₁, ..., b_k}, c)** : Associe un intervalle global *a* à un ensemble de sous-intervalles optionnels {b₁, ..., b_k}. Si *a* est présent, exactement *c* sous-intervalles parmi {b₁, ..., b_k} doivent être présents, et ces *c* sous-intervalles sélectionnés doivent commencer et finir en même temps que *a*. Dans sa version standard (*c* = 1), cette contrainte modélise le choix d'une ressource unique parmi plusieurs alternatives.

2.3.3 Ressources disjonctives et cumulatives

- **Non-chevauchement (noOverlap)** : Pour modéliser une ressource disjonctive (comme un travailleur qui ne peut exécuter qu'une seule tâche à la fois), on regroupe les variables d'intervalles affectées à cette ressource dans une contrainte **noOverlap**. Elle garantit que deux intervalles présents ne se chevauchent pas dans le temps.

2.4 Modélisation CP monolithique pour le MS-RCPSP-AF

Afin de modéliser la flexibilité d'affectation sans préemption de la tâche globale, nous développons un modèle par programmation par contraintes fondé sur la discrétisation interne des activités en parties unitaires.

2.4.1 Ensembles et paramètres

Le modèle réutilise les ensembles de base introduits au Chapitre 1 ($\mathcal{A}, \mathcal{L}, \mathcal{W}, \mathcal{CR}, E$) et y ajoute l'ensemble suivant :

- $V_i = \{1, \dots, p_i\}$: Ensemble des parties de durée unitaire de l'activité $i \in \mathcal{A}$, indexées par v .

Nous définissons le paramètre supplémentaire suivant :

- N_l : Nombre total de travailleurs maîtrisant la compétence l dans l'ensemble $\mathcal{W}$.

2.4.2 Variables de décision

Le modèle utilise des variables d'intervalles à plusieurs niveaux :

- $C_{\max} \geq 0$: Variable entière représentant la date de fin globale du projet.
- act_i : Variable d'intervalle représentant l'exécution globale de l'activité i , de taille p_i .
- $par_{i,v}$: Variable d'intervalle de durée unitaire représentant la v -ème partie de l'activité i (pour $v \in V_i$).
- $awo_{w,i,l,v}$: Variable d'intervalle optionnelle de durée unitaire représentant l'affectation du travailleur w exerçant la compétence l pour la v -ème partie de l'activité i .

2.4.3 Formulation mathématique

Le modèle CP monolithique s'écrit de la manière suivante :

$$\min \quad C_{\max} \quad (14)$$

s.t.

$$C_{\max} \geq act_i.\text{end} \quad \forall i \in \mathcal{A} \quad (15)$$

$$\text{span}(act_i, \{par_{i,v} \mid v \in V_i\}) \quad \forall i \in \mathcal{A} \quad (16)$$

$$\text{startAtEnd}(par_{i,v+1}, par_{i,v}) \quad \forall i \in \mathcal{A}, \forall v \in \{1, \dots, p_i - 1\} \quad (17)$$

$$\text{endBeforeStart}(act_i, act_m) \quad \forall (i, m) \in E \quad (18)$$

$$\sum_{i \in \mathcal{A}} \text{pulse}(act_i, b_{i,k}) \leq B_k \quad \forall k \in \mathcal{CR} \quad (19)$$

$$\text{noOverlap}(\{awo_{w,i,l,v} \mid \forall i \in \mathcal{A}, \forall l \in \mathcal{L}, \forall v \in V_i\}) \quad \forall w \in \mathcal{W} \quad (20)$$

$$\text{alternative}(par_{i,v}, \{awo_{w,i,l,v} \mid \forall w \in \mathcal{W} \text{ s.t. } m_{w,l} = 1\}, a_{i,l}) \quad \forall i \in \mathcal{A}, \forall v \in V_i, \forall l \in \mathcal{L} \quad (21)$$

$$\text{alternative}(par_{i,v}, \{awo_{w,i,l,v} \mid \forall w \in \mathcal{W}, \forall l \in \mathcal{L}\}, q_i) \quad \forall i \in \mathcal{A}, \forall v \in V_i \quad (22)$$

2.4.4 Explication des contraintes

- L'équation (15) définit le makespan à minimiser comme supérieur ou égal à la fin de chaque activité globale act_i .
- La contrainte (16) lie l'activité globale act_i à l'ensemble de ses sous-intervalles unitaires $par_{i,v}$.
- L'équation (17) assure la contiguïté temporelle (non-préemption) en forçant la fin de la partie v à coïncider avec le début de la partie $v + 1$.
- La contrainte de précédence (18) assure les relations d'ordre du projet entre les activités globales.
- L'équation (19) restreint l'usage simultané des ressources matérielles cumulatives à leur capacité B_k .
- L'équation (20) garantit la disjonction de chaque travailleur : un opérateur w ne peut pas travailler sur plusieurs tâches ou plusieurs compétences en même temps.
- La contrainte alternative (21) modélise la flexibilité : pour chaque partie unitaire v de l'activité i et pour chaque compétence l , elle force la sélection d'exactly $a_{i,l}$ intervalles optionnels d'affectation parmi les travailleurs qualifiés.
- L'équation (22) fait de même pour satisfaire la contrainte d'effectif physique minimum global q_i sur chaque partie unitaire.

2.4.5 Contraintes redondantes de capacité globale

Afin d'accélérer la propagation et d'aider le solveur à détecter plus rapidement les surcharges de capacité de main-d'œuvre, nous introduisons des enveloppes cumulatives globales redondantes :

$$wU = \sum_{i \in \mathcal{A}} \text{pulse}(\text{act}_i, q_i) \leq |\mathcal{W}| \quad (23)$$

$$sU_l = \sum_{i \in \mathcal{A}} \text{pulse}(\text{act}_i, a_{i,l}) \leq N_l \quad \forall l \in \mathcal{L} \quad (24)$$

Ces enveloppes ne modifient pas l'espace des solutions, mais renforcent les mécanismes de filtrage en évaluant la demande de main-d'œuvre au niveau des activités globales act_i plutôt que sur chaque partie unitaire.

2.5 Variante : Modélisation par Contrainte de Cardinalité Globale (GCC)

La modélisation CP présentée ci-dessus engendre un nombre massif de variables d'intervalles optionnelles ($awo_{w,i,l,v}$), dépassant fréquemment 10 000 à 25 000 variables sur des instances de taille moyenne. Ce volume considérable crée un surcoût mémoire important et alourdit le travail du moteur de propagation à chaque nœud de l'arbre de recherche.

Pour pallier cette limitation et tenter de fluidifier la résolution, nous avons conçu une modélisation alternative reposant sur la Contrainte de Cardinalité Globale (GCC, ou *Global Cardinality Constraint*). L'objectif principal est de réduire le nombre de variables d'intervalles en éliminant leur indexation par la compétence l . L'affectation est ainsi déportée sur des variables entières $SW_{w,i,v}$ associées à la primitive `distribute` de CP Optimizer, qui permet d'imposer des fréquences d'apparition exactes des valeurs d'un tableau de variables.

2.5.1 Variables de décision du modèle GCC

Pour chaque tâche $i \in \mathcal{A}$, chaque partie unitaire $v \in V_i$, et chaque travailleur $w \in \mathcal{W}$, nous définissons :

- Une variable entière d'affectation de compétence, notée $SW_{w,i,v}$, représentant la compétence que le travailleur w utilise pour réaliser la partie v de la tâche i .
- Le domaine de cette variable $SW_{w,i,v}$ est restreint à :

$$\mathcal{D}(SW_{w,i,v}) = \{0\} \cup (\mathcal{MS}_w \cap \mathcal{SA}_i) \quad (25)$$

où :

- La valeur 0 signifie que le travailleur w n'est pas affecté à la tâche i à l'instant v .
- Un entier $l \geq 1$ représente l'indice de la compétence exercée.
- $\mathcal{MS}_w$ est l'ensemble des compétences maîtrisées par le travailleur w .
- $\mathcal{SA}_i$ est l'ensemble des compétences exigées par la tâche i .

- Une variable d'intervalle optionnelle $AW_{w,i,v}$ représentant la présence du travailleur w sur la partie v de la tâche i . Cette variable d'intervalle est synchronisée avec la variable entière $SW_{w,i,v}$ par la contrainte logique :

$$\text{presence_of}(AW_{w,i,v}) \iff (SW_{w,i,v} > 0) \quad (26)$$

2.5.2 Formulation du modèle GCC et contraintes d'affectation

La modélisation par contrainte de cardinalité globale repose sur trois contraintes qui complètent les contraintes temporelles du modèle monolithique (équations 16–19) :

$$\text{distribute}(\text{cards}, \{SW_{w,i,v} \mid w \in \mathcal{W}\}, \text{values}) \quad \forall i \in \mathcal{A}, \forall v \in V_i \quad (27)$$

$$\text{alternative}\left(\text{par}_{i,v}, \{AW_{w,i,v} \mid w \in \mathcal{W}\}, \sum_{l \in \mathcal{L}} a_{i,l}\right) \quad \forall i \in \mathcal{A}, \forall v \in V_i \quad (28)$$

$$\text{noOverlap}(\{AW_{w,i,v} \mid i \in \mathcal{A}, v \in V_i\}) \quad \forall w \in \mathcal{W} \quad (29)$$

où $\text{values} = [0, 1, 2, \dots, |\mathcal{L}|]$ désigne les valeurs d'affectation possibles (0 pour l'inactivité, $l \geq 1$ pour la compétence l), et cards est le vecteur de cardinalités défini par :

$$\text{cards}[0] = |\mathcal{W}| - \sum_{l \in \mathcal{L}} a_{i,l} \quad \text{et} \quad \text{cards}[l] = a_{i,l} \quad \forall l \in \mathcal{L} \quad (30)$$

- La contrainte (27) assure le respect des exigences en compétences : pour chaque partie unitaire v de la tâche i , elle force exactement $a_{i,l}$ travailleurs à prendre la valeur l dans $\{SW_{w,i,v} \mid w \in \mathcal{W}\}$, et $|\mathcal{W}| - \sum_{l \in \mathcal{L}} a_{i,l}$ travailleurs à prendre la valeur 0 (non affectés).
- La contrainte (28) lie temporellement les parties unitaires $\text{par}_{i,v}$ aux variables d'intervalles d'affectation $AW_{w,i,v}$: si $\text{par}_{i,v}$ est présente, exactement $\sum_{l \in \mathcal{L}} a_{i,l}$ travailleurs sont actifs et partagent ses dates de début et de fin.
- La contrainte (29) garantit la disjonction de chaque travailleur w : ses intervalles d'affectation $AW_{w,i,v}$ ne peuvent pas se chevaucher dans le temps.

2.6 Contraintes de bris de symétrie

Ces formulations monolithiques en CP présentent des symétries de permutation. Si plusieurs travailleurs possèdent les mêmes compétences, les solutions consistant à permuter ces travailleurs sur des tâches identiques sont équivalentes pour le makespan, mais obligent le solveur à explorer des branches inutilement.

Pour limiter ces symétries, nous introduisons une contrainte forçant la conservation de l'affectation des travailleurs entre les unités de temps, sauf lorsqu'une nouvelle activité débute. Autrement dit, si une partie unitaire $\text{par}_{i,v}$ ($v \geq 1$) est active alors qu'aucune tâche ne débute à cet instant,

l'affectation des travailleurs pour la compétence l sur cette partie v est forcée d'être identique à celle de la partie précédente $v - 1$.

La contrainte s'exprime ainsi :

$$\text{IfThen}(PaS - \text{pulse}(par_{i,v}, 1) \leq -1, awo_{w,i,l,v} = awo_{w,i,l,v-1}) \quad \forall l \in \mathcal{L}, \forall w \in \mathcal{W}, \forall i \in \mathcal{A}, \forall v \in V_i \setminus \{0\} \quad (31)$$

avec :

$$PaS = \sum_{i \in \mathcal{A}} \text{pulse}(par_{i,0}, 1) \quad (32)$$

qui vaut zéro partout sauf lorsqu'au moins une tâche débute (c'est-à-dire lors de l'exécution de la première partie unitaire $par_{i,0}$ de n'importe quelle activité i).

Ainsi, si une partie unitaire $par_{i,v}$ ($v \geq 1$) est active alors qu'aucune tâche ne débute à cet instant ($PaS = 0$, donc $PaS - \text{pulse}(par_{i,v}, 1) = -1$), l'affectation des travailleurs pour la compétence l sur cette partie v est forcée d'être identique à celle de la partie précédente $v - 1$.

2.7 Limites des modèles CP monolithiques

Ces formulations monolithiques en CP offrent un cadre de modélisation lisible grâce aux primitives de CP Optimizer (`interval`, `alternative`) ainsi que des solutions tout-en-un gérant à la fois les contraintes d'ordonnancement et d'affectation. Cependant, leur mise en œuvre sur des instances de taille moyenne à grande se heurte à des limites de passage à l'échelle. Cette section en analyse les facteurs majeurs.

2.7.1 Explosion dimensionnelle du nombre de variables

La flexibilité d'affectation autorise les travailleurs à se relayer au cours de l'exécution d'une même activité. Pour capturer ce comportement sans perdre la continuité temporelle de la tâche, le modèle fragmente chaque activité $i \in \mathcal{A}$ de durée p_i en p_i sous-activités unitaires de durée 1, notées $par_{i,v}$. Prenons le cas de la première formulation :

1. **Variables d'intervalles unitaires** : Pour chaque tâche i , on génère p_i variables d'intervalles obligatoires $par_{i,v}$.
2. **Variables d'intervalles optionnelles d'affectation** : Pour chaque sous-activité unitaire $par_{i,v}$, pour chaque compétence l exigée et pour chaque travailleur w maîtrisant cette compétence ($m_{w,l} = 1$), on crée une variable d'intervalle optionnelle $awo_{w,i,l,v}$.

Le nombre de variables d'intervalles optionnelles à gérer s'exprime par la relation :

$$N_{opt} = \sum_{i \in \mathcal{A}} p_i \sum_{l \in \mathcal{L}} a_{i,l} \sum_{w \in \mathcal{W}} m_{w,l} \quad (33)$$

Pour une instance standard de taille moyenne (30 tâches, 5 compétences, 16 travailleurs polyvalents et des durées $p_i \approx 15$), N_{opt} dépasse 10 000 à 25 000 variables optionnelles. Cette dimensionnelle

sature la mémoire vive et pèse sur le moteur de propagation de CP Optimizer, qui doit maintenir la cohérence de ces contraintes d’alternatives.

2.7.2 Perte de la structure globale d’ordonnancement et faiblesse de la propagation

La fragmentation des activités en intervalles unitaires réduit l’efficacité de la structure des tâches. Au lieu de manipuler des blocs temporels (ce pour quoi CP Optimizer est optimisé via des propagateurs spécifiques comme le *edge-finding*), le solveur traite des micro-intervalles chaînés.

Cette structure présente deux faiblesses :

- **Faiblesse du filtrage des ressources :** Les contraintes cumulatives de capacité de ressources (exprimées par `pulse` ou `alwaysIn`) sont propagées localement sur chaque micro-intervalle de durée 1. Le solveur peine à projeter les conflits de ressources sur le long terme car il ne perçoit plus la tâche comme un bloc consommant des ressources sur toute sa durée.
- **Surcharge du réseau de contraintes de précédence :** Pour assurer la contiguïté temporelle de l’activité, le modèle impose des contraintes de séquençement strictes entre chaque partie unitaire successive. Ce chaînage crée un réseau de précédences dense qui ralentit la propagation des fenêtres de temps.

2.7.3 Symétries et inefficacité des contraintes de bris de symétrie

Le MS-RCPSP-AF souffre d’une forte symétrie de permutation des travailleurs. Si deux travailleurs possèdent le même profil de compétences, toute permutation de ces travailleurs produit des plannings équivalents pour le makespan.

Dans ces modèles, cette symétrie est accentuée par la discrétisation temporelle : le solveur explore des permutations de travailleurs d’une unité de temps à l’autre au sein d’une même tâche. Par exemple, affecter un travailleur sur la première moitié d’une tâche et un autre sur la seconde moitié, ou vice-versa, est traité comme deux branches distinctes, alors que le makespan global reste inchangé.

L’ajout de contraintes de bris de symétrie vise à élaguer ces branches redondantes. Cependant, comme observé expérimentalement au chapitre 4, ces contraintes imposent un coût de filtrage trop lourd, conduisant à des dégradations de performances et à des timeouts supplémentaires.

Ces limites mettent en évidence l’intérêt d’envisager des méthodes de décomposition comme la LBBD. En séparant l’ordonnancement de l’affectation des ressources (déléguée à un sous-problème), la LBBD s’affranchit de la discrétisation temporelle et de l’explosion du nombre de variables associée.

3 Décomposition de Benders basée sur la logique

Ce chapitre présente la méthode de décomposition de Benders basée sur la Logique (LBBD, pour *Logic-Based Benders Decomposition*). Cette approche sépare les décisions temporelles d’ordonnancement (dates d’exécution des tâches) des décisions d’affectation (attribution des travailleurs à chaque période).

Afin de mieux introduire les fondements et les apports de la LBBD, rappelons brièvement le principe de la décomposition de Benders classique [26]. Elle partitionne les variables d’un problème en deux sous-ensembles résolus de manière itérative : un problème maître et un sous-problème. La version classique impose que le sous-problème soit un programme linéaire continu afin d’exploiter les variables duales pour construire des coupes d’optimalité ou de faisabilité. La décomposition de Benders basée sur la logique (LBBD) [10, 11] étend ce principe aux cas où le sous-problème contient des variables entières ou des structures combinatoires discrètes (résolu par programmation par contraintes ou algorithmes de graphes). Les coupes ne proviennent plus de la dualité linéaire continue, mais de conditions logiques formulées à partir de la preuve d’infaisabilité ou d’optimalité du sous-problème.

La LBBD est fréquemment appliquée aux problèmes couplant ordonnancement et allocation de ressources [12]. Dans le cas du MSPSP, les premières approches utilisaient un schéma classique : un problème maître en MILP pour l’affectation globale et un sous-problème en CP pour l’ordonnancement temporel [17, 18, 27]. Pour améliorer la convergence, Yi-fan et al. [13] ont proposé d’inverser cette structure en confiant l’ordonnancement à un problème maître CP et l’affectation au sous-problème. Récemment, Juvin et al. [1] ont étendu ce schéma inverse au problème avec flexibilité d’affectation (MS-RCPSP-AF).

3.1 Principes de la Décomposition de Benders basée sur la Logique appliquée au MS-RCPSP-AF

La décomposition de Benders basée sur la logique découple la planification des tâches de l’affectation nominative des opérateurs. La résolution repose sur une boucle itérative entre un problème maître et un sous-problème : le problème maître (formulé en CP) génère un ordonnancement minimisant le makespan sous une relaxation des ressources et l’ensemble des coupes déjà accumulées. Cet ordonnancement est ensuite transmis au sous-problème, qui évalue de façon indépendante à chaque période si la polyvalence des travailleurs permet de couvrir les exigences en compétences (modélisé par un graphe biparti et résolu par flot maximum). Si l’affectation est réalisable, la solution est immédiatement prouvée optimale : d’une part, le problème maître garantit que le makespan obtenu est minimal parmi tous les ordonnancements compatibles avec les coupes accumulées ; d’autre part, le sous-problème confirme l’existence d’une affectation valide des travailleurs pour cet ordonnancement, en fournissant une assignation concrète des travailleurs à chaque période. L’algorithme s’arrête alors. En cas d’infaisabilité, le sous-problème identifie la sous-structure critique responsable du conflit d’affectation et génère une coupe combinatoire réinjectée dans le problème maître pour éliminer cette configuration lors des itérations futures.

Pour résumer, la décomposition s'articule ainsi :

- **Le problème maître (CP)** : Il planifie les activités sur un horizon temporel continu en respectant les contraintes de précédences, les contraintes de capacité cumulatives de main-d'œuvre globales ainsi que les coupes accumulées. Il ignore l'affectation nominative des travailleurs, mais fournit un ordonnancement global des tâches.
- **Le sous-problème d'affectation (Flot Max)** : Pour un ordonnancement fixé par le problème maître, il vérifie de manière indépendante pour chaque période t s'il existe une affectation valide des travailleurs pour couvrir les besoins en compétences des activités actives. En cas d'infaisabilité, les coupes logiques générées correspondent à des ensembles critiques ou interdits (*forbidden sets*) [28, 29].

3.2 Problème maître (Ordonnancement)

Le problème maître est une version relaxée du premier modèle CP monolithique présenté au Chapitre 2. Il se concentre sur l'aspect temporel de l'ordonnancement global des tâches. Il ignore l'affectation nominative des opérateurs, mais intègre des contraintes de capacité cumulative globale (enveloppes de main-d'œuvre) pour éviter de générer des plannings manifestement irréalisables.

3.2.1 Formulation mathématique

Soit Ω l'ensemble des coupes de Benders générées et injectées dynamiquement au cours des itérations. Le problème maître s'exprime sous la forme du modèle CP suivant :

$$\min \quad C_{\max} \tag{34}$$

s.t.

$$C_{\max} \geq act_i.end \quad \forall i \in \mathcal{A} \tag{35}$$

$$endBeforeStart(act_i, act_m) \quad \forall (i, m) \in E \tag{36}$$

$$\sum_{i \in \mathcal{A}} pulse(act_i, b_{i,k}) \leq B_k \quad \forall k \in \mathcal{CR} \tag{37}$$

$$\sum_{i \in \mathcal{A}} pulse(act_i, q_i) \leq |\mathcal{W}| \tag{38}$$

$$\sum_{i \in \mathcal{A}} pulse(act_i, a_{i,l}) \leq N_l \quad \forall l \in \mathcal{L} \tag{39}$$

$$\Omega \tag{40}$$

Les équations (35) à (39) correspondent aux contraintes d'ordonnancement et aux contraintes cumulatives globales (le besoin en effectif total ne doit pas dépasser le nombre d'opérateurs $|\mathcal{W}|$, et la demande cumulée par compétence ne doit pas dépasser le nombre d'opérateurs qualifiés N_l). La contrainte (40) représente les coupes générées par les sous-problèmes d'affectation pour interdire les conflits de compétences détectés.

3.3 Sous-problème d'affectation

À chaque itération, le solveur résout le problème maître et obtient un ordonnancement partiel où les dates de début et de fin de chaque activité globale act_i sont fixées. À partir de cet ordonnancement, nous déterminons pour chaque période $t \in \mathcal{T}$ l'ensemble des activités actives :

$$\mathcal{A}(t) = \{i \in \mathcal{A} \mid act_i.start \leq t < act_i.end\} \quad (41)$$

Pour chaque période t , la demande totale pour la compétence l , notée $D_l(t)$, est la somme des besoins des tâches actives à cet instant :

$$D_l(t) = \sum_{i \in \mathcal{A}(t)} a_{i,l} \quad \forall l \in \mathcal{L} \quad (42)$$

La validation de la faisabilité consiste à vérifier s'il existe, pour chaque période t , une affectation des travailleurs $w \in \mathcal{W}$ aux compétences $l \in \mathcal{L}$ telle que la demande $D_l(t)$ soit satisfaite pour toute compétence et que chaque travailleur ne soit affecté qu'au plus à une seule compétence.

Pour modéliser ce sous-problème d'affectation à chaque période t , deux approches sont envisageables. La première consiste à formuler un programme linéaire de décision de faisabilité en relâchant les contraintes d'intégralité binaire sur les variables d'affectation. Soit $x_{w,l} \geq 0$ la variable de décision continue représentant la fraction de temps que le travailleur w consacre à la compétence l durant la période t . Le problème se formule ainsi :

$$\min \quad 0 \quad (43)$$

$$\text{s.t.} \quad \sum_{l \in \mathcal{L}} x_{w,l} \leq 1 \quad \forall w \in \mathcal{W} \quad (44)$$

$$\sum_{w \in \mathcal{W}} x_{w,l} \cdot m_{w,l} = D_l(t) \quad \forall l \in \mathcal{L} \quad (45)$$

$$x_{w,l} \geq 0 \quad \forall w \in \mathcal{W}, \forall l \in \mathcal{L} \quad (46)$$

La contrainte (44) garantit qu'un travailleur n'est pas affecté au-delà de sa capacité temporelle unitaire, tandis que la contrainte (45) assure la satisfaction exacte de la demande en compétences requise. L'objectif est simplement de trouver une solution admissible (coût nul).

Pour éviter d'appeler un solveur de programmation linéaire générique (simplexe ou points intérieurs) à chaque étape de la recherche, nous utilisons plutôt la seconde approche : la modélisation sous forme de flot maximum dans un réseau biparti. Ce formalisme permet d'appliquer des algorithmes de graphes spécialisés plus rapides.

3.3.1 Formulation sous forme de flot maximum (Max-Flow)

Puisque les périodes sont indépendantes une fois le temps fixé, le problème d'affectation pour une période t peut être modélisé comme un problème de flot maximum dans un réseau de transport biparti.

Nous construisons un graphe orienté $G(V, A)$ composé de :

- Une source s et un puits p .
- Un nœud pour chaque travailleur $w \in \mathcal{W}$.
- Un nœud pour chaque couple tâche-compétence (i, l) actif à l'instant t .

Les arcs et leurs capacités associées sont définis comme suit :

- Un arc orienté de la source s vers chaque nœud de travailleur w avec une capacité unitaire (1), garantissant qu'un travailleur ne peut effectuer qu'une tâche à la fois.
- Un arc orienté de chaque nœud de travailleur w vers chaque nœud de tâche-compétence (i, l) si et seulement si le travailleur maîtrise la compétence requise ($m_{w,l} = 1$). La capacité de cet arc est infinie.
- Un arc orienté de chaque nœud de tâche-compétence (i, l) vers le puits p avec une capacité égale à la demande requise $a_{i,l}$.

Le problème de flot maximum peut se formuler comme un programme linéaire :

$$\max \quad \sum_{w \in \mathcal{W}} f_{s,w} \quad (47)$$

$$\text{s.t.} \quad f_{s,w} \leq 1 \quad \forall w \in \mathcal{W} \quad (48)$$

$$f_{s,w} - \sum_{\substack{i \in \mathcal{A}(t) \\ l \in \mathcal{L}}} f_{w,(i,l)} = 0 \quad \forall w \in \mathcal{W} \quad (49)$$

$$\sum_{\substack{w \in \mathcal{W} \\ m_{w,l}=1}} f_{w,(i,l)} - f_{(i,l),p} = 0 \quad \forall i \in \mathcal{A}(t), \forall l \in \mathcal{L} \quad (50)$$

$$f_{(i,l),p} \leq a_{i,l} \quad \forall i \in \mathcal{A}(t), \forall l \in \mathcal{L} \quad (51)$$

$$f \geq 0 \quad (52)$$

La contrainte (48) borne le flot sortant de la source à 1 par travailleur, garantissant qu'un travailleur ne peut être mobilisé que pour une seule compétence à la fois. Les contraintes (49) et (50) assurent la conservation du flot respectivement aux nœuds travailleurs et aux nœuds tâche-compétence. Enfin, la contrainte (51) borne le flot entrant au puits par la demande $a_{i,l}$ de chaque couple actif.

Si la valeur du flot maximum est strictement inférieure à la demande totale $\sum_{l \in \mathcal{L}} D_l(t)$, alors l'affectation est irréalisable à l'instant t . Le solveur doit alors générer une coupe de Benders pour exclure cet ordonnancement.

3.4 Génération des coupes de Benders

Lorsqu'une infaisabilité est détectée à une période t , c'est-à-dire lorsque le flot maximum calculé est strictement inférieur à la demande totale $\sum_{l \in \mathcal{L}} D_l(t)$, il est nécessaire d'en identifier la cause structurelle avant de pouvoir faire progresser le problème maître. Cette infaisabilité ne signifie pas

que le problème global est sans solution : elle indique uniquement que l’ordonnancement proposé par le problème maître crée, à cet instant t , une surcharge localisée sur un sous-ensemble de compétences que la main-d’œuvre disponible ne peut satisfaire.

Pour exploiter cette information, nous identifions le sous-ensemble critique de compétences à l’origine du conflit à l’aide d’une coupe minimale extraite du graphe biparti. Ce sous-ensemble est ensuite traduit en une contrainte cumulative globale, appelée coupe de Benders, que l’on réinjecte dans le problème maître afin d’interdire toute configuration d’ordonnancement reproduisant ce conflit lors des itérations futures.

3.4.1 Recherche de coupe minimale sur le graphe biparti

Nous exploitons directement le théorème du flot maximum / coupe minimale (*Max-Flow / Min-Cut*) [30] sur le réseau biparti orienté construit avec la bibliothèque **NetworkX**. Ce théorème établit que, dans tout réseau de flot, la valeur du flot maximum entre une source et un puits est égale à la capacité de la coupe minimale séparant ces deux nœuds. De plus, la partition obtenue identifie le sous-ensemble de compétences le plus petit possible à l’origine de l’infaisabilité.

Lorsqu’à une période t , le flot maximum calculé est strictement inférieur à la demande totale ($\text{FlotMax} < \sum_{l \in \mathcal{L}} D_l(t)$), la coupe minimale partitionne les nœuds du graphe en deux sous-ensembles disjoints : la partie atteignable (notée S_{att} , contenant la source s) et la partie non atteignable (notée $T_{\text{non-att}}$, contenant le puits p).

Soit $S \subseteq \mathcal{L}$ le sous-ensemble des compétences situées dans la partie non atteignable ($T_{\text{non-att}}$). La capacité de la coupe minimale est donnée par la somme des capacités des arcs coupés :

$$\text{Capacite}(S_{\text{att}}, T_{\text{non-att}}) = |W_S| + \sum_{l \notin S} D_l(t) \quad (53)$$

où $W_S \subseteq \mathcal{W}$ représente l’ensemble des travailleurs situés dans la partie non atteignable ($T_{\text{non-att}}$), c’est-à-dire les travailleurs capables d’exercer au moins une compétence de S . En effet, aucun travailleur de S_{att} ne peut maîtriser une compétence de S : un tel arc $w \rightarrow (i, l)$ traverserait la coupe avec une capacité infinie, ce qui est impossible pour une coupe minimale finie.

L’infaisabilité de l’affectation à l’instant t implique que la capacité de cette coupe est strictement inférieure à la demande totale exigée par les tâches actives :

$$|W_S| + \sum_{l \notin S} D_l(t) < \sum_{l \in \mathcal{L}} D_l(t) \implies |W_S| < \sum_{l \in S} D_l(t) \quad (54)$$

Cette inégalité isole le goulet d’étranglement : le nombre de travailleurs $|W_S|$ maîtrisant les compétences du sous-ensemble S est strictement insuffisant pour couvrir la demande $\sum_{l \in S} D_l(t)$ requise à l’instant t .

3.4.2 Formulation de la coupe injectée

Pour interdire cette situation dans le problème maître, nous injectons la contrainte cumulative suivante :

$$\sum_{i \in \mathcal{A}} \sum_{l \in S} \text{pulse}(act_i, a_{i,l}) \leq |W_S| \quad (55)$$

Cette coupe indique que la somme des demandes en compétences appartenant à S pour toutes les activités s'exécutant simultanément ne doit pas dépasser le nombre de travailleurs qualifiés $|W_S|$. Cette contrainte est formulée sous forme de fonctions cumulatives et de propagateurs temporels, ce qui permet au problème maître de décaler dans le temps certaines activités en conflit lors de la recherche.

4 Évaluation Expérimentale

Dans ce chapitre, nous présentons l'évaluation expérimentale de l'ensemble des approches modélisées : le modèle de programmation par contraintes monolithiques classiques, la formulation basée sur la contrainte de cardinalité globale, la méthode de décomposition de Benders basée sur la logique (LBBD) ainsi que leurs variantes avec contraintes de bris de symétrie. L'objectif est d'analyser leurs performances en termes de temps de résolution ainsi que leurs comportements sur différents jeux de données.

4.1 Protocole expérimental et instances de test

4.1.1 Environnement de test

Les expérimentations ont été menées sur une machine équipée d'un processeur Intel Xeon W6-3423 cadencé à 2.1 GHz et de 62 Go de mémoire RAM, sur la distribution Linux : Ubuntu. Les solveurs utilisés sont :

- **IBM ILOG CP Optimizer** (version 22.1) pour la résolution du modèle CP monolithique et du Problème Maître CP dans la décomposition.
- L'implémentation de l'algorithme de flot maximum d'Edmonds-Karp du package python **NetworkX** pour résoudre le sous-problème d'affectation à chaque période.

Pour chaque instance, la limite de temps de calcul a été fixée à **500 secondes**.

4.1.2 Premier jeu de données : Set 1a de la bibliothèque MSPSP-InstLib

L'évaluation repose sur un ensemble de 215 instances de référence issues de la bibliothèque publique MSPSP-InstLib¹ introduite par Young et al. [8]. Ce dataset, nommé « Set 1a », a été conçu à l'aide d'un générateur dérivé des travaux d'Almeida et al. [5] et converti au format DataZinc (.dzn). Ces instances se caractérisent par :

- Un nombre d'activités $n = 20$.
- Un effectif de travailleurs $m \in \{10, 13, 15, 20, 25, 30\}$.
- Un facteur de complexité réseau ($nc \in \{1.5, 1.8, 2.1\}$), qui régit la densité du graphe.
- Un facteur de compétences ($sf \in \{0, 0.5, 0.75, 1.0\}$), caractérisant la diversité et le degré de maîtrise des compétences ($sf = 0$ correspond à un facteur de compétences variable et aléatoire).

Ce jeu d'instances offre une diversité de configurations de contraintes et de ressources pour l'analyse comparative.

¹<https://github.com/youngkd/MSPSP-InstLib>

4.1.3 Autre jeu de données : Subset 1 de MSLIB

Bien que le jeu d'instances de la MSPSP-InstLib permette d'obtenir des comparaisons utiles et de valider les modèles, sa taille ($n = 20$ tâches) limite sa portée pratique pour représenter des problématiques industrielles réelles.

Pour évaluer la robustesse des approches sur d'autres structures, nous avons également choisi de tester nos modèles sur un autre jeu de données de taille supérieure : la bibliothèque MSLIB² introduite par Snauwaert et Vanhoucke [4].

Cette bibliothèque a été conçue par l'équipe du professeur Mario Vanhoucke à l'Université de Gand. MSLIB se structure en 5 grands jeux de données : les jeux MSLIB1 à MSLIB4 (artificiels) et le jeu MSLIB5 (regroupant 18 cas réels). Nous utilisons le jeu MSLIB1, qui comporte 6 600 instances de taille supérieure :

- Un nombre d'activités fixe $n = 32$ (30 tâches et 2 tâches fictives).
- Un nombre de compétences fixe $|\mathcal{L}| = 4$.
- Un effectif de travailleurs $|\mathcal{W}|$ variable, allant de 4 à 312.

Construction des sous-ensembles (Subsets) : Résoudre les 6 600 instances représenterait un coût de calcul important. Pour obtenir un échantillon calculable, nous avons partitionné le jeu en 33 sous-ensembles représentatifs de 200 instances.

Cette partition a été construite par un découpage par round-robin stratifié : Les 6 600 instances ont été triées par ordre croissant du nombre de travailleurs $|\mathcal{W}|$. Les instances ont ensuite été distribuées cycliquement dans les 33 sous-ensembles.

Ce processus permet d'assurer que chaque sous-ensemble présente une distribution similaire des effectifs de ressources (médiane $|\mathcal{W}| \approx 49$, écart-type ≈ 54). Pour nos expérimentations, nous utilisons le Subset 1 (200 instances).

4.2 Comparaison des approches sur le Set 1a de MSPSP-InstLib

Les résultats obtenus sur les 215 instances du Set 1a pour les différentes méthodes de résolution (CP Classique, CP Classique avec bris de symétrie, CP Distribute, CP Distribute avec bris de symétrie et la Décomposition de Benders basée sur la Logique) sont résumés dans le tableau 1.

Table 1: Synthèse globale des résultats de résolution sur le Set 1a (Time Limit = 500s)

Approche	Inst. résolues	Succès (%)	Temps moy. (s)	Temps hors timeouts (s)
CP Classique	169 / 215	78.6 %	151.31	56.38
CP Classique + Symétrie	164 / 215	76.3 %	159.39	53.46
CP GCC (Distribute)	157 / 215	73.0 %	152.86	24.60
CP GCC + Symétrie	152 / 215	70.7 %	165.94	27.46
Décomposition de Benders (LBBD)	209 / 215	97.2 %	24.70	11.05

²<https://github.com/MarioVanhoucke/MSLIB-Multi-Skilled-Resource-Constrained-Project-Library>

4.2.1 Analyse des taux de résolution et des temps de calcul

La Décomposition de Benders basée sur la Logique (LBBD) obtient de meilleurs résultats que les approches monolithiques. Elle résout 209 instances sur 215 (97.2 %), ne subissant que 6 timeouts. Parmi ces 209 instances résolues, 165 l'ont été en moins d'une seconde (79 %), et 183 en moins de 10 secondes, ce qui illustre la vitesse de convergence la décomposition. La LBBD résout par ailleurs 40 instances qu'aucune approche CP monolithique n'a pu résoudre dans la limite impartie.

Le modèle CP classique échoue à prouver l'optimalité sur 46 instances (21.4 % de timeouts) et le modèle CP GCC (Distribute) sur 58 instances (27.0 % de timeouts). Le passage au modèle CP GCC réduit le nombre de variables d'intervalles optionnelles mais diminue l'efficacité de la propagation sur les instances difficiles, ce qui se traduit par un taux de résolution plus faible (73.0 %). Cependant, le temps moyen hors timeouts est nettement réduit (24.60 s contre 56.38 s pour le CP Classique), ce qui suggère que le modèle basé sur la GCC est plus rapide sur les instances faciles que la formulation classique mais peine à résoudre les instances les plus difficiles.

L'ajout de contraintes de bris de symétrie dégrade systématiquement les performances : le modèle CP classique avec contrainte de bris de symétrie résout 5 instances de moins que le modèle CP classique, et le modèle CP GCC avec contrainte de bris de symétrie 5 de moins également que le modèle CP GCC. Ce résultat contre-intuitif s'explique par le coût de filtrage des contraintes conditionnelles, qui n'est pas compensé par la réduction de l'espace de recherche.

La décomposition de Benders résout les instances en un temps moyen de 24.70 secondes, avec une moyenne de seulement 1,6 itérations entre le problème maître et le sous-problème d'affectation. De plus, toutes les instances résolues l'ont été en 5 itérations ou moins.

4.2.2 Comparaison directe sur les instances faciles (communes)

Pour comparer les approches sans le biais introduit par les timeouts, nous avons isolé les 132 instances résolues à l'optimalité par l'ensemble des cinq méthodes, et calculé le temps moyen de résolution sur cet ensemble commun :

- **CP Classique : 24.47 s**
- **CP Classique avec contrainte de bris de symétrie : 16.42 s**
- **CP GCC : 8.54 s**
- **CP GCC avec contrainte de bris de symétrie : 15.03 s**
- **Décomposition de Benders : 0.84 s**

Sur ces instances communes, la décomposition de Benders reste de loin la plus rapide (0.84 s), soit environ 30 fois plus rapide que le modèle CP classique. Les modèles GCC obtiennent des temps intermédiaires (8.54 s et 15.03 s), bénéficiant d'une propagation allégée par la réduction du nombre de variables d'intervalles optionnelles.

4.3 Visualisations graphiques pour le Set 1a

La figure 2 illustre le profil de performance des 5 méthodes sur le Set 1a.

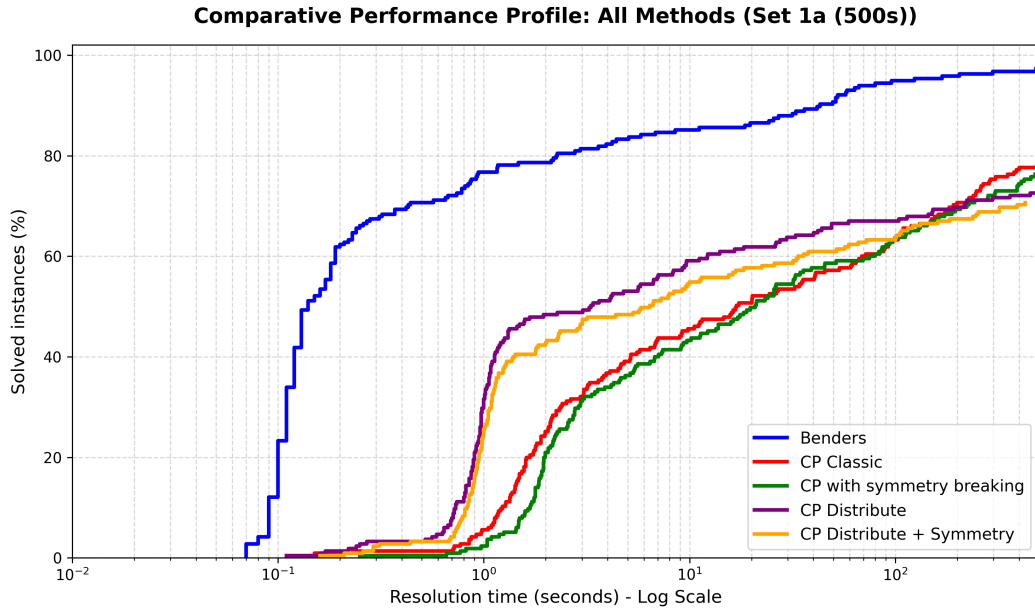


Figure 2: Profil de performance cumulatif unifié : Comparaison du taux de résolution (%) en fonction du temps de calcul (échelle logarithmique) sur le Set 1a (215 instances - 500s).

4.3.1 Impact du nombre de travailleurs sur les performances de résolution

Parmi les différentes caractéristiques structurelles des instances du Set 1a, l'analyse expérimentale révèle que l'effectif de travailleurs m est le facteur qui exerce l'impact le plus déterminant sur les performances de résolution. La figure 3 illustre l'évolution du temps moyen global de résolution en fonction de m pour l'ensemble des méthodes.

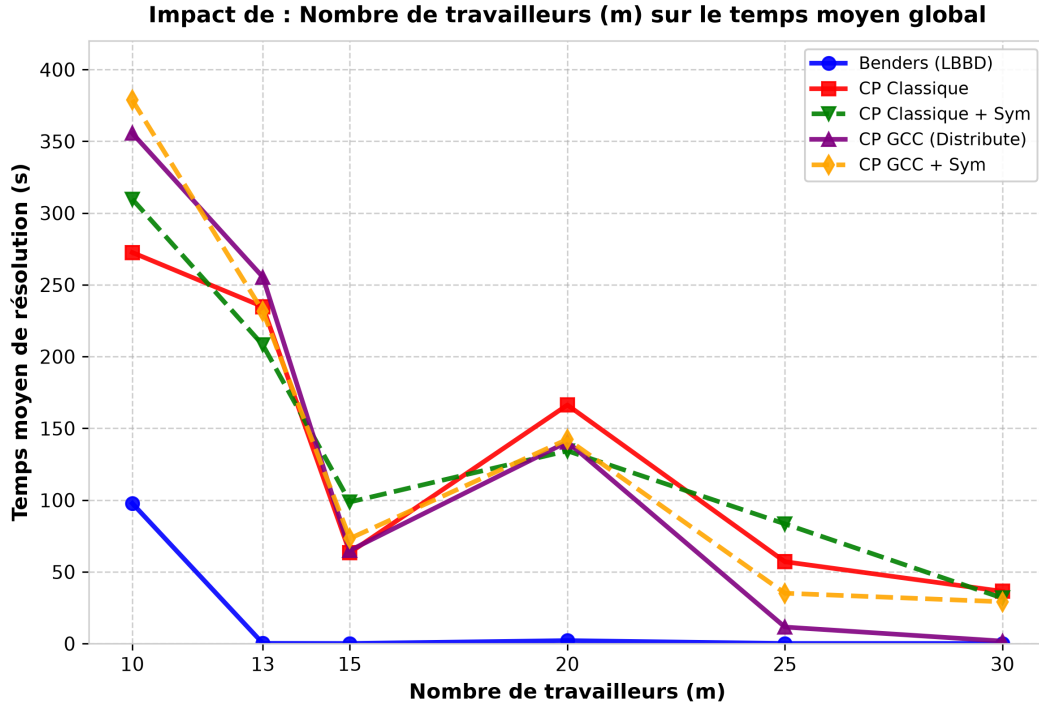


Figure 3: Impact du nombre de travailleurs (m) sur le temps moyen global de résolution (Set 1a).

On constate de manière très marquée que plus la quantité de ressources multi-compétences augmente, plus les performances s’améliorent. Par exemple, pour la méthode CP GCC, le temps moyen de résolution s’effondre de 378.8 secondes pour $m = 10$ à seulement 1.8 seconde pour $m = 30$.

Sur le plan théorique, la disponibilité d’une main-d’œuvre polyvalente plus nombreuse diminue la rareté des ressources et désature les contraintes d’affectation sur chaque période unitaire. Cela réduit fortement le nombre de conflits d’allocation simultanés et facilite la découverte de plannings réalisables pour le solveur.

4.4 Résultats et comparaison sur le Subset 1 de MSLIB1

Les performances de l’ensemble des cinq méthodes de résolution (CP Classique, CP Classique avec bris de symétrie, CP GCC, CP GCC avec bris de symétrie, et LBBD) ont été évaluées sur le sous-ensemble de 200 instances **Subset 1** de la bibliothèque MSLIB1 avec une limite de temps de 500 secondes. Les résultats sont présentés dans le tableau 2.

Table 2: Synthèse globale des résultats de résolution sur MSLIB1 (Subset 1, 200 instances, Time Limit = 500s)

Approche	Inst. résolues	Succès (%)	Temps moy. (s)	Temps hors timeouts (s)
CP Classique	172 / 200	86.0 %	78.64	10.02
CP Classique + Symétrie	164 / 200	82.0 %	100.16	12.38
CP GCC (Distribute)	161 / 200	80.5 %	105.72	10.19
CP GCC + Symétrie	162 / 200	81.0 %	102.27	8.96
Décomposition de Benders (LBBD)	188 / 200	94.0 %	32.62	2.78

4.4.1 Analyse des taux de résolution et des temps de calcul

La Décomposition de Benders basée sur la Logique (LBBD) confirme sa supériorité sur ce jeu d'instances plus vaste ($n = 32$ tâches). Elle résout 188 instances sur 200 (94.0 %), ne concédant que 12 timeouts. Parmi les instances résolues, 170 l'ont été en moins d'une seconde (90.4 %), ce qui illustre la rapidité de la convergence. De plus, la LBBD résout 16 instances qu'aucune méthode CP monolithique n'a pu résoudre, sans qu'aucune instance résolue par le CP ne lui échappe.

Pour les modèles CP monolithiques, le modèle CP Classique résout 172 instances (86.0 %), surpassant le modèle CP GCC (161 instances, 80.5 %). L'ajout de contraintes de bris de symétrie dégrade les performances du CP Classique (164 résolutions contre 172), mais apporte une légère amélioration sur le CP GCC (162 résolutions contre 161, avec un temps hors timeouts réduit à 8.96 s).

La LBBD résout l'ensemble des instances en un temps moyen de 2.78 secondes (hors timeouts), avec un nombre d'itérations moyen de seulement 1.23 entre le problème maître et le sous-problème d'affectation (le nombre maximal d'itérations observé étant de 6).

4.4.2 Comparaison directe sur les instances faciles (communes)

En isolant les 153 instances résolues à l'optimalité par l'ensemble des cinq méthodes, nous comparons les temps moyens de résolution sans le biais des timeouts :

- **CP Classique : 3.22 s**
- **CP Classique avec contrainte de bris de symétrie : 6.67 s**
- **CP GCC : 9.37 s**
- **CP GCC avec contrainte de bris de symétrie : 6.65 s**
- **Décomposition de Benders : 0.37 s**

Sur ces instances communes, la décomposition de Benders est environ 9 fois plus rapide que le CP Classique (0.37 s contre 3.22 s) et 25 fois plus rapide que le CP GCC.

4.4.3 Comparaison du comportement MSLIB vs Set 1a

La confrontation des deux jeux de données révèle quatre enseignements principaux :

1. **Le CP Classique est plus rapide hors timeouts sur MSLIB1 :** Le temps moyen hors timeouts chute de 56.38 s (Set 1a) à 10.02 s (Subset 1). L'effectif moyen de travailleurs plus élevé ($m \approx 49$) pour un nombre de compétences restreint ($|\mathcal{L}| = 4$) rend l'affectation moins tendue, favorisant la propagation.
2. **Écart réduit pour le modèle GCC :** L'avantage du modèle GCC sur les instances faciles s'estompe sur MSLIB1 (10.19 s contre 10.02 s pour le CP Classique), car la réduction du nombre de variables d'intervalles apporte moins de bénéfices quand la capacité en ressources n'est pas le facteur le plus restrictif.

3. **Sensibilité au bris de symétrie** : Si les contraintes de bris de symétrie dégradent le CP Classique sur les deux jeux, elles deviennent légèrement bénéfiques pour le CP GCC sur MSLIB1 (+1 instance résolue, 8.96 s contre 10.19 s), les symétries de permutation étant plus marquées avec des effectifs plus larges.
4. **Convergence encore plus rapide pour Benders** : Le nombre moyen d'itérations passe de 1.6 sur le Set 1a à 1.23 sur Subset 1, confirmant que l'efficacité de la coupe minimale ne s'altère pas lorsque le nombre de tâches augmente.

4.5 Visualisations graphiques pour le Subset 1

La figure 4 présente le profil de performance cumulé unifié pour l'ensemble des méthodes sur le Subset 1 de MSLIB1.

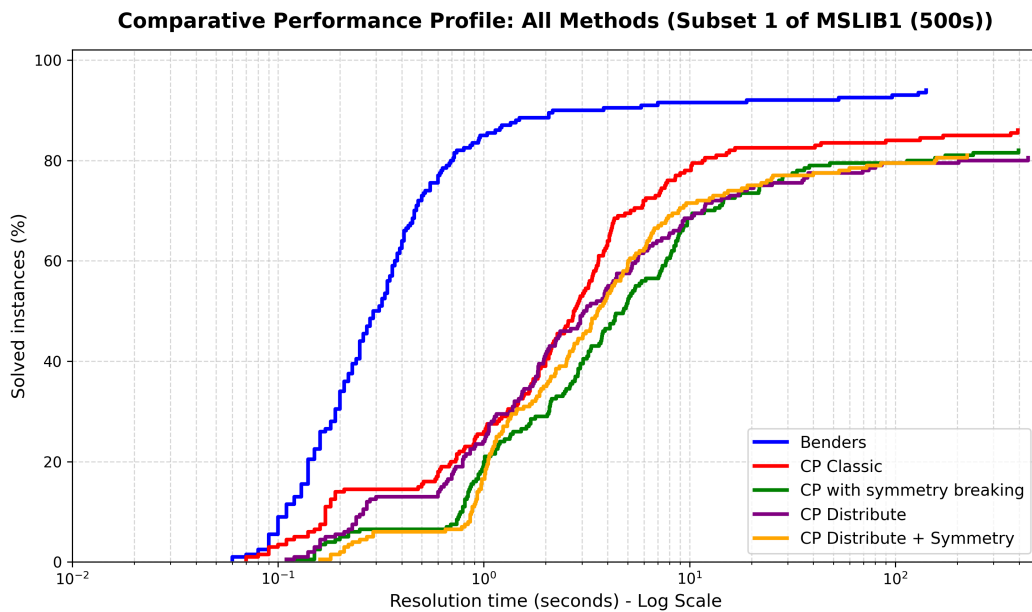


Figure 4: Profil de performance cumulé unifié : Comparaison du taux de résolution (%) en fonction du temps de calcul (échelle logarithmique) sur MSLIB1 (Subset 1 - 200 instances - 500s).

Conclusion et Perspectives

Ce travail de stage a permis de concevoir, d'implémenter et d'évaluer une chaîne complète d'approches de résolution exactes pour le problème d'ordonnancement de projet sous contraintes de ressources multi-compétences avec flexibilité d'affectation (MS-RCPSP-AF). Après avoir formulé des modèles monolithiques en programmation par contraintes (modèle classique avec variables d'intervalles unitaires et variante reposant sur la contrainte de cardinalité globale, ainsi que leurs versions intégrant des contraintes de bris de symétrie), nous avons analysé leurs verrous combinatoires majeurs, notamment l'explosion du nombre de variables d'intervalles optionnelles et l'affaiblissement du filtrage *edge-finding* sur des tâches fragmentées. Pour lever ces limitations, nous avons modélisé et implémenté une méthode de décomposition de Benders basée sur la logique, découplant la planification temporelle (confiée à un problème maître CP) de la vérification des affectations nominatives des opérateurs (déléguée à un sous-problème de flot maximum par période). En cas d'infaisabilité, l'extraction de la coupe minimale sur le réseau biparti via la bibliothèque **NetworkX** isole le sous-ensemble critique de compétences responsable du conflit et génère une coupe cumulative réinjectée dynamiquement dans le problème maître. Les évaluations expérimentales menées sur les jeux de référence valident la domination de cette approche : sur le Set 1a ($n = 20$), la LBBDD atteint un taux de succès de 97.2 % (contre 78.6 % pour le CP classique) en 1.6 itération en moyenne et s'avère 29 fois plus rapide sur les instances communes ; sur le jeu étendu MSLIB1 ($n = 32$), elle résout 94.0 % des instances en 1.23 itération en moyenne et 2.78 secondes hors timeouts.

Sur le plan personnel, ce stage m'a permis d'approfondir et de mettre en pratique les concepts théoriques et algorithmiques abordés lors de l'UE *Algorithmique Avancée* suivie au cours du premier semestre du Master 1 Informatique parcours IAFA à l'Université de Toulouse. Cette immersion au sein du LAAS a constitué une opportunité privilégiée pour découvrir le monde de la recherche académique et pour me projeter concrètement dans le rôle et le travail quotidien d'un chercheur. Échanger, débattre et collaborer au quotidien avec les chercheurs et doctorants de l'équipe ROC s'est avéré être une expérience humaine et scientifique particulièrement enrichissante, confortant pleinement mes aspirations professionnelles.

Au-delà des performances obtenues, ce travail ouvre plusieurs perspectives prometteuses. Sur le plan de la valorisation scientifique, un article de recherche est en cours de rédaction : il permettra de publier et de diffuser les formulations théoriques des modèles développés ainsi que l'ensemble des résultats expérimentaux obtenus. Sur le plan algorithmique, il serait intéressant d'explorer de nouvelles formulations alternatives en programmation par contraintes pour tenter de rivaliser avec la LBBDD, d'envisager d'autres paradigmes de résolution ainsi que d'autres méthodes hybrides de décomposition. Enfin, la richesse de cette thématique et la diversité des défis scientifiques ouverts dépassent largement le cadre d'un stage : ce sujet fera ainsi l'objet d'un travail de thèse de doctorat, qui étendra l'application de la décomposition de Benders basée sur la logique à un autre problème d'ordonnancement sous contraintes de ressources complexes.

References

- [1] C. Juvin, C. Artigues, P. Lopez, and R. Brux, “Logic-based benders method for multi-skill resource-constrained project scheduling with allocation and skill flexibility,” *Preprint submitted to Elsevier*, 2026.
- [2] O. Bellenguez-Morineau and E. Néron, “A branch-and-bound method for solving multi-skill project scheduling problem,” *RAIRO-Operations Research*, vol. 41, no. 2, pp. 155–170, 2007.
- [3] ———, “Multi-mode and multi-skill project scheduling problem,” in *Resource-Constrained Project Scheduling: Models, Algorithms, Extensions and Applications*, C. Artigues, S. Demassey, and E. Néron, Eds. Wiley Online Library, 2008, pp. 149–160.
- [4] J. Snauwaert and M. Vanhoucke, “A classification and new benchmark instances for the multi-skilled resource-constrained project scheduling problem,” *European Journal of Operational Research*, vol. 307, no. 1, pp. 1–19, 2023.
- [5] B. F. Almeida, I. Correia, and F. Saldanha-da-Gama, “Modeling frameworks for the multi-skill resource-constrained project scheduling problem: A theoretical and empirical comparison,” *International Transactions in Operational Research*, vol. 26, no. 3, pp. 946–967, 2019.
- [6] I. Correia and F. Saldanha-da-Gama, “A modeling framework for project staffing and scheduling problems,” in *Handbook on Project Management and Scheduling Vol. 1*, C. Schwindt and J. Zimmermann, Eds. Springer, 2015, pp. 547–564.
- [7] C. Montoya, O. Bellenguez-Morineau, E. Pinson, and D. Rivreau, “Branch-and-price approach for the multi-skill project scheduling problem,” *Optimization Letters*, vol. 8, pp. 1721–1734, 2014.
- [8] K. D. Young, T. Feydy, and A. Schutt, “Constraint programming applied to the multi-skill project scheduling problem,” in *Principles and Practice of Constraint Programming: 23rd International Conference (CP 2017), Melbourne, VIC, Australia, August 28–September 1, 2017, Proceedings 23*. Springer, 2017, pp. 308–317.
- [9] O. Polo-Mejía, C. Artigues, P. Lopez, and V. Basini, “Mixed-integer/linear and constraint programming approaches for activity scheduling in a nuclear research facility,” *International Journal of Production Research*, vol. 58, no. 23, pp. 7149–7166, 2020.
- [10] J. N. Hooker, *Logic-based methods for optimization: Combining optimization and constraint satisfaction*. New York: John Wiley and Sons, 2000.
- [11] ———, “Logic-based benders decomposition for large-scale optimization,” *Large scale optimization in supply chains and smart manufacturing: Theory and applications*, pp. 1–26, 2019.
- [12] A. A. Cire, E. Coban, and J. N. Hooker, “Logic-based benders decomposition for planning and scheduling: A computational analysis,” *The Knowledge Engineering Review*, vol. 31, no. 5, pp. 440–451, 2016.

- [13] W. Yi-fan, S. Fu-quan, L. Shi-xin, and C. Di, “A new method to solve project scheduling problems with multi-skilled workforce constraints,” in *2013 25th Chinese Control and Decision Conference (CCDC)*. IEEE, 2013, pp. 2408–2412.
- [14] R. M. Karp, *Reducibility among combinatorial problems*. Springer, 2010.
- [15] C. H. Papadimitriou and K. Steiglitz, *Combinatorial optimization: Algorithms and complexity*. Courier Corporation, 1998.
- [16] E. L. Demeulemeester and W. S. Herroelen, *Project scheduling: A research handbook*. Springer Science & Business Media, 2006, vol. 49.
- [17] H. Li and K. Womer, “Scheduling projects with multi-skilled personnel by a hybrid MILP/CP Benders decomposition algorithm,” *Journal of Scheduling*, vol. 12, no. 3, pp. 281–298, 2009.
- [18] H. Li, “Benders decomposition approach for project scheduling with multi-purpose resources,” in *Handbook on Project Management and Scheduling Vol. 1*, C. Schwindt and J. Zimmermann, Eds. Springer, 2015, pp. 587–601.
- [19] O. Polo-Mejía, C. Artigues, P. Lopez, L. Mönch, and V. Basini, “Heuristic and metaheuristic methods for the multi-skill project scheduling problem with partial preemption,” *International Transactions in Operational Research*, vol. 30, no. 2, pp. 858–891, 2023.
- [20] O. Polo Mejía, “Operational research approach for optimising the operations of a nuclear research laboratory,” Ph.D. dissertation, Institut National des Sciences Appliquées de Toulouse, Sep 2019. [Online]. Available: <https://hal.science/tel-02309284>
- [21] H. Fahimi and C.-G. Quimper, “Linear-time filtering algorithms for the disjunctive constraint,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 28(1), 2014.
- [22] P. Vilím, “Edge finding filtering algorithm for discrete cumulative resources in $\mathcal{O}(kn \log n)$,” in *International Conference on Principles and Practice of Constraint Programming (CP)*. Springer, 2009, pp. 802–816.
- [23] Y. Ouellet and C.-G. Quimper, “A $O(n \log^2 n)$ checker and $O(n^2 \log n)$ filtering algorithm for the energetic reasoning,” in *International Conference on the Integration of Constraint Programming, Artificial Intelligence, and Operations Research (CPAIOR)*. Springer, 2018, pp. 477–494.
- [24] J. Carlier and E. Néron, “Computing redundant resources for the resource constrained project scheduling problem,” *European Journal of Operational Research*, vol. 176, no. 3, pp. 1452–1463, 2007.
- [25] P. Baptiste and N. Bonifas, “Redundant cumulative constraints to compute preemptive bounds,” *Discrete Applied Mathematics*, vol. 234, pp. 168–177, 2018.
- [26] J. Benders, “Partitioning procedures for solving mixed-variables programming problems,” *Numerische Mathematik*, vol. 4, no. 1, pp. 238–252, 1962.

- [27] A. Nasirian, L. Zhang, A. M. Costa, and B. Abbasi, “Multiskilled workforce staffing and scheduling: A logic-based Benders’ decomposition approach,” *European Journal of Operational Research*, vol. 323, no. 1, pp. 20–33, 2025.
- [28] M. Lombardi and M. Milano, “A min-flow algorithm for minimal critical set detection in resource constrained project scheduling,” *Artificial Intelligence*, vol. 182, pp. 58–67, 2012.
- [29] F. Stork and M. Uetz, “On the generation of circuits and minimal forbidden sets,” *Mathematical programming*, vol. 102, pp. 185–203, 2005.
- [30] L. R. Ford and D. R. Fulkerson, “Maximal flow through a network,” *Canadian Journal of Mathematics*, vol. 8, pp. 399–404, 1956.